

Chapitre 5 : Agréger et restructurer avec pandas

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python pour la science des données · **Fil rouge** : les prix des denrées sur les marchés du Sénégal · **Version** : 2026-10-23

Fiche du chapitre

Objectifs (OS4)	A la fin du chapitre, l'apprenant sait : 1) regrouper des lignes et calculer une statistique par groupe (`groupby`) ; 2) calculer plusieurs statistiques à la fois, avec des noms clairs (`agg`) ; 3) comparer chaque ligne à son groupe (`transform`) ; 4) réunir deux tableaux par une colonne commune (`merge`) et contrôler le résultat ; 5) empiler des tableaux (`concat`) ; 6) construire un tableau croisé (`pivot_table`) ; 7) passer du format large au format long (`melt`) et inversement (`pivot`).
Prérequis	Chapitre 4 (charger, sélectionner, convertir, nettoyer)
Durée	🟢 Débutant : 8 h · 🔵 Standard : 6 h · 🟣 Expert : 4 h (atelier compris)
Matériel	Google Colab ; les relevés du PAM et le fichier des marchés (déjà fournis)
Livrable	Le notebook de l'atelier 5 : un classeur d'indicateurs par région et par denrée, commenté

Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Ouvrez un nouveau notebook Colab, `chapitre5-agregation`, envoyez-y le fichier des prix et le fichier des marchés du PAM, et tapez les exemples dans l'ordre.

---

1. Des 51 854 relevés aux réponses

Au chapitre 4, vous avez appris à rendre un tableau **propre**. Mais un tableau propre de 51 854 lignes ne répond encore à aucune question. Les questions de la vraie vie sont des questions **par groupe** :

- Quel est le prix médian du mil **dans chaque région** ?
- Quelle région est la plus chère **pour chaque denrée** ?
- Comment le prix du riz a-t-il évolué **d'année en année** ?
- Un marché vend-il plus cher que **les autres marchés du même mois** ?

Répondre à ces questions, c'est **agréger** : résumer beaucoup de lignes en quelques nombres. Et souvent, il faut aussi **restructurer** : réunir deux fichiers, ou mettre les denrées en colonnes pour lire un tableau d'un coup d'œil. Ce chapitre donne les six outils de pandas pour cela.

2. Préparer les relevés

On reprend les gestes du chapitre 4 : garder les prix au kilo, choisir et renommer les colonnes, convertir la date.

```
import pandas as pd

prix = pd.read_csv("wfp_food_prices_sen.csv")
prix = prix[prix["unit"] == "KG"].copy()
prix = prix[["date", "admin1", "market_id", "market", "category", "commodity", "price"]]
prix = prix.rename(columns={"admin1": "region", "market": "marche", "category": "categorie",
                            "commodity": "denree", "price": "prix"})
prix["date"] = pd.to_datetime(prix["date"])
prix["annee"] = prix["date"].dt.year
print(prix.shape)
prix.head(3)
```

```
(51854, 8)
        date region  market_id  marche          categorie            denree    prix  annee
0 2000-01-15  Dakar        447  Tilene  cereals and tubers  Maize (imported)  149.00   2000
1 2000-01-15  Dakar        447  Tilene  cereals and tubers            Millet  124.75   2000
2 2000-01-15  Dakar        447  Tilene  cereals and tubers   Rice (imported)  227.50   2000
```

On garde `market_id`, le numéro du marché : il servira à la section 6 pour retrouver les marchés dans un autre fichier.

3. Regrouper avec `groupby`

3.1 Une statistique par groupe

```
prix.groupby("denree")["prix"].mean().round(1).sort_values(ascending=False).head(5)
```

```
denree
Fonio                   1250.0
Groundnuts (shelled)     874.1
Bananas (local)          838.6
Beans (niebe)            829.9
Watermelons              780.2
```

Morceau	Ce qu'il veut dire
`prix.groupby("denree")`	Range les lignes en **groupes**, un par denrée différente
`["prix"]`	Dans chaque groupe, on ne regarde que la colonne `prix`
`.mean()`	La moyenne de chaque groupe. Le résultat est une `Series` : une valeur par denrée, la denrée servant d'**index**
`.round(1).sort_values(ascending=False).head(5)`	Arrondir, trier du plus cher au moins cher, garder les 5 premiers (chapitre 4)

🟢 `groupby` travaille toujours en trois temps, qu'on appelle **diviser, appliquer, combiner** (*split, apply, combine*) :

1. Diviser
les 51 854 lignes sont rangées en 26 paquets, un par denrée (Mil, Riz importé, Fonio...)

2. Appliquer
dans chaque paquet, on calcule la moyenne de la colonne `prix`

3. Combiner
les 26 moyennes sont rassemblées dans un seul résultat, indexé par la denrée

⚠️ Le fonio arrive en tête... mais regardez combien de relevés se cachent derrière cette moyenne :

```python
groupes = prix.groupby("denree")
print(type(groupes))
print(groupes.ngroups, "groupes")
groupes.get_group("Fonio")
```

```
<class 'pandas.api.typing.DataFrameGroupBy'>
26 groupes
            date    region  market_id     marche           categorie denree    prix  annee
50767 2026-06-15   Kaolack        697  N'Doffane  cereals and tubers  Fonio  1300.0   2026
50813 2026-06-15  Kedougou        419   Kedougou  cereals and tubers  Fonio  1200.0   2026
```

`groupby` seul ne calcule rien : il prépare les groupes (`ngroups` les compte, `get_group` en montre un). Le fonio n'a que **deux** relevés, en juin 2026 : sa moyenne ne vaut pas grand-chose. **Règle** : à côté d'une moyenne par groupe, regardez toujours **combien de lignes** il y a dans chaque groupe.

3.2 Compter : `size`

Pour la suite, on se concentre sur l'année 2025, la dernière année complète :

```
recents = prix[prix["annee"] == 2025]
recents.groupby("region").size().sort_values().head(4)
```

```
region
Sedhiou         43
Kaffrine       111
Saint Louis    161
Ziguinchor     180
```

`size()` compte les lignes de chaque groupe. Sédhiou n'a que 43 relevés en 2025 : ses statistiques seront plus fragiles que celles des autres régions.

🔵 `count()` compte, lui, les valeurs **non manquantes** d'une colonne. Ici, aucun prix ne manque, donc `groupby("region")["prix"].count()` donne les mêmes nombres ; sur un tableau avec des trous, les deux diffèrent.

> 🧨 **Essayez** : `recents.groupby("regions")["prix"].mean()` (avec un s), puis `recents.groupby("region")["marche"].mean()`. Messages : `KeyError: 'regions'` (la colonne n'existe pas) et `TypeError: dtype 'str' does not support operation 'mean'` (on ne fait pas la moyenne de noms de marchés). Avec pandas 2, le second message est différent, mais c'est aussi une `TypeError`. Annulez.

3.3 Plusieurs clés

On peut regrouper sur **plusieurs** colonnes à la fois : une ligne par couple (région, denrée).

```
moy = recents.groupby(["region", "denree"])["prix"].mean().round(1)
print(moy.head(4))
print(len(moy), "couples (région, denrée)")
moy.loc[("Dakar", "Millet")]
```

```
region  denree
Dakar   Beans (niebe)             971.6
        Groundnuts (shelled)      857.2
        Groundnuts (unshelled)    929.7
        Maize (imported)          308.9
Name: prix, dtype: float64
145 couples (région, denrée)
np.float64(292.0)
```

Morceau	Ce qu'il veut dire
`groupby(["region", "denree"])`	Une liste de colonnes : un groupe par **combinaison** région et denrée
L'index à deux niveaux	pandas affiche la région une seule fois, puis ses denrées en dessous : c'est un **index multiple** (*MultiIndex*)

Morceau	Ce qu'il veut dire
145 couples	Et non 14 x 11 = 154 : certaines denrées ne sont pas relevées dans certaines régions. Un couple sans ligne n'apparaît pas
`moy.loc[("Dakar", "Millet")]`	Une valeur précise, avec un **tuple** des deux étiquettes. `np.float64(292.0)` : un nombre NumPy (chapitre 3), qui vaut 292 F

Un index multiple est pratique pour lire, mais moins pour la suite des calculs. `as_index=False` garde les clés comme des **colonnes ordinaires** :

```
tableau = recents.groupby(["region", "denree"], as_index=False)["prix"].mean()
tableau.head(3)
```

```
  region                 denree        prix
0  Dakar           Beans (niebe)  971.629444
1  Dakar    Groundnuts (shelled)  857.176111
2  Dakar  Groundnuts (unshelled)  929.687500
```

On obtient un `DataFrame` ordinaire, qu'on peut filtrer, trier, enregistrer. `.reset_index()` après coup fait la même chose.

4. Plusieurs statistiques avec `agg`

4.1 Une liste de statistiques

```
recents.groupby("denree")["prix"].agg(["count", "mean", "median", "min", "max"]).round(1).head(5)
```

```
                     count   mean  median    min     max
denree
Beans (niebe)          376  848.6   879.2  375.0  1250.0
Groundnuts (shelled)   394  887.2   850.0  400.0  1500.0
Groundnuts (unshelled) 302  456.2   384.2  250.0  1150.0
Maize (imported)       250  288.9   300.0  185.0   350.0
Maize (local)          410  286.4   300.0  150.0   680.0
```

`agg` (*aggregate*, « agréger ») reçoit une liste de statistiques, désignées par leur nom en texte, et donne une **colonne par statistique**. En une ligne, le nombre de relevés, la moyenne, la médiane et l'étendue des prix de chaque denrée.

4.2 Des colonnes différentes, des noms clairs

Souvent, on veut des statistiques sur **différentes** colonnes, avec des noms en français. C'est l'**agrégation nommée** :

```python
resume = recents.groupby("region").agg(
    releves=("prix", "size"),
    marches=("marche", "nunique"),
    denrees=("denree", "nunique"),
    prix_median=("prix", "median"),
)
resume.sort_values("releves", ascending=False).head(5)
```

```
            releves  marches  denrees  prix_median
region
Fatick          448        6       11       350.00
Thies           404        4       11       400.00
Tambacounda     397        4       11       350.00
Diourbel        374        4       11       411.25
Matam           331        3       11       400.00
```

Morceau	Ce qu'il veut dire
`releves=("prix", "size")`	Une nouvelle colonne `releves` : la statistique `size` sur la colonne `prix`. Forme générale : `nom_choisi=(colonne, statistique)`
`marches=("marche", "nunique")`	Le nombre de marchés **différents** de la région
`prix_median=("prix", "median")`	La médiane de tous les prix de la région, toutes denrées confondues

C'est la forme la plus lisible : le nom de chaque colonne dit ce qu'elle contient.

⚠️ `prix_median` mélange toutes les denrées : une région qui relève beaucoup d'arachide (900 F) aura une médiane plus haute qu'une région qui relève surtout du mil (300 F), sans être plus chère. Pour comparer les régions honnêtement, il faut comparer **denrée par denrée** : ce sera la section 5 et l'atelier.

4.3 Sa propre statistique

`agg` accepte aussi une **fonction** à vous, qui reçoit la `Series` d'un groupe et renvoie un nombre :

```python
def etendue(serie):
    return serie.max() - serie.min()


recents.groupby("denree")["prix"].agg(["min", "max", etendue]).sort_values("etendue",
         ascending=False).head(3)
```

```
                        min     max  etendue
denree
Groundnuts (shelled)    400.0  1500.0   1100.0
Groundnuts (unshelled)  250.0  1150.0    900.0
Beans (niebe)           375.0  1250.0    875.0
```

La fonction est passée **sans guillemets** et **sans parenthèses** (c'est la fonction elle-même, pas son résultat) ; la colonne prend son nom. L'arachide décortiquée a le plus grand écart : de 400 à 1 500 F le kilo selon le marché et le mois.

> 🧨 **Essayez** : `recents.groupby("region")["prix"].agg(["moyenne"])` . Message : `AttributeError: 'SeriesGroupBy' object has no attribute 'moyenne'` . Les noms en texte sont les noms anglais des méthodes de pandas : `"mean"` , `"median"` , `"sum"` , `"min"` , `"max"` , `"count"` , `"size"` , `"nunique"` , `"std"` . Annulez.

5. Comparer chaque ligne à son groupe : `transform`

`agg` résume un groupe en **une** ligne. `transform` fait l'inverse : il calcule la statistique du groupe et la **recopie sur chaque ligne** du groupe. On peut alors comparer chaque ligne à son groupe.

Question : un marché vend-il le riz importé plus cher ou moins cher que la moyenne des marchés **du même mois** ?

```
riz = recents[recents["denree"] == "Rice (imported)"].copy()
riz["moyenne_du_mois"] = riz.groupby("date")["prix"].transform("mean")
riz["ecart_pct"] = ((riz["prix"] / riz["moyenne_du_mois"] - 1) * 100).round(1)
riz[["date", "marche", "prix", "moyenne_du_mois", "ecart_pct"]].sort_values("ecart_pct").head(3)
```

```
           date      marche   prix  moyenne_du_mois  ecart_pct
45832 2025-04-15     Diakhao  227.5       410.942895      -44.6
48176 2025-11-15  Saint-Maur  290.0       357.533333      -18.9
47804 2025-10-15       Louga  307.5       364.515152      -15.6
```

Morceau	Ce qu'il veut dire
`riz.groupby("date")["prix"]`	Les relevés de riz, groupés par mois (une date par mois dans ce fichier : le 15)
`.transform("mean")`	La moyenne du mois, **répétée** sur chaque ligne de ce mois : le résultat a autant de lignes que `riz` , on peut donc en faire une colonne
`riz["prix"] / riz["moyenne_du_mois"] - 1`	L'écart relatif : `-0.446` signifie 44,6 % de moins que la moyenne du mois

Comparer au **même mois** évite de confondre un marché bon marché et un mois bon marché. Le relevé de Diakhao (227,5 F, 45 % sous la moyenne) est si éloigné des autres qu'il mérite une vérification : erreur de saisie, ou riz d'une autre qualité ?

On peut ensuite résumer ces écarts par région :

```
riz.groupby("region")["ecart_pct"].mean().round(1).sort_values()
```

```
region
Kolda          -5.5
Ziguinchor     -5.2
Saint Louis    -5.1
Sedhiou        -4.3
Kaolack        -3.0
Thies          -2.7
Fatick         -1.1
Louga          -0.8
Kaffrine       -0.4
Diourbel        0.5
Tambacounda     1.0
Matam           1.7
Kedougou        4.5
Dakar          19.8
```

En 2025, à Dakar, le riz importé se vend en moyenne **20 % plus cher** que dans l'ensemble des marchés le même mois.

`agg`	`transform`
Une ligne par groupe	**Autant** de lignes qu'au départ
Pour résumer (« le prix médian par région »)	Pour comparer chaque ligne à son groupe (« ce relevé est-il cher pour son mois ? »)

Au chapitre 4 (atelier, étape 5), vous avez déjà utilisé `transform("median")` pour repérer les prix au sac : chaque prix était comparé à la médiane de **sa** denrée.

6. Réunir deux tableaux : `merge`

6.1 Le fichier des marchés

Le PAM fournit un second fichier, la liste des marchés :

```
marches = pd.read_csv("wfp_markets_sen.csv")
print(marches.shape)
marches.head(3)
```

```
(79, 7)
   market_id   market countryiso3       admin1   admin2  latitude  longitude
0        405    Bakel         SEN  Tambacounda    Bakel     14.90     -12.46
1        406   Bambey         SEN     Diourbel   Bambey     14.70     -16.46
2        407  Bignona         SEN   Ziguinchor  Bignona     12.81     -16.23
```

79 marchés, avec leur région (`admin1`), leur département (`admin2`) et leurs coordonnées. Les relevés, eux, ne portent que sur 67 marchés (chapitre 4, exercice 1). Les deux fichiers ont une colonne en commun : `market_id` . C'est la **clé** qui permet de les réunir.

6.2 La jointure interne

Comptons les relevés de chaque marché, puis réunissons ce compte avec la liste des marchés :

```
nb = prix.groupby("market_id").size().rename("releves").reset_index()
print(len(nb), "marchés avec des relevés")
avec = pd.merge(marches, nb, on="market_id")
print(len(avec))
avec[["market", "admin1", "admin2", "releves"]].head(3)
```

```
67 marchés avec des relevés
67
    market       admin1   admin2  releves
0    Bakel  Tambacounda    Bakel     1023
1   Bambey     Diourbel   Bambey     1162
2  Bignona   Ziguinchor  Bignona     1070
```

Morceau	Ce qu'il veut dire
`.size().rename("releves").reset_index()`	Le nombre de relevés par marché, transformé en tableau à deux colonnes : `market_id` et `releves`
`pd.merge(marches, nb, on="market_id")`	Pour chaque ligne de `marches`, cherche la ligne de `nb` qui a le **même** `market_id`, et colle leurs colonnes côte à côte
67 lignes, pas 79	Par défaut, `merge` ne garde que les clés présentes **dans les deux** tableaux : c'est la **jointure interne** (`how="inner"`). Les 12 marchés sans relevé ont disparu, sans message

6.3 Les quatre jointures, et `indicator`

Disparaître sans message est dangereux. Pour savoir ce qui s'est passé, on garde **toutes** les lignes du tableau de gauche et on demande un **indicateur** :

```
tous = pd.merge(marches, nb, on="market_id", how="left", indicator=True)
print(tous["_merge"].value_counts())
tous[tous["_merge"] == "left_only"][["market_id", "market", "admin1"]].head(4)
```

```
_merge
both          67
left_only     12
right_only     0
Name: count, dtype: int64
    market_id      market       admin1
64       5839   Thiadiaye        Thies
65       5840  Wak Ngouna      Kaolack
67       5844     Ranerou        Matam
68       5845      Ndioum  Saint Louis
```

`how=`	**Garde…**	**Ici**
`"inner"` (par défaut)	les clés présentes **des deux côtés**	67 marchés
`"left"`	**toutes** les lignes de gauche ; `NaN` à droite quand il n'y a pas de correspondance	79 marchés, dont 12 avec `releves` vide
`"right"`	toutes les lignes de droite	67
`"outer"`	tout, des deux côtés	79

`indicator=True` ajoute la colonne `_merge` : `both` (trouvé des deux côtés), `left_only` (seulement à gauche), `right_only` (seulement à droite). Ici, 12 marchés figurent dans la liste mais n'ont **aucun** relevé : marchés ajoutés récemment, ou abandonnés ? C'est le genre de question qu'une jointure contrôlée fait apparaître.

🟢 Comme pour `groupby`, une image aide : `merge` est un **recoupement de deux listes**. On cherche chaque nom de la première liste dans la seconde ; `inner` garde ceux qu'on trouve, `left` garde toute la première liste en laissant des cases vides.

6.4 Ajouter des informations : les noms en français

`merge` sert aussi à **enrichir** un tableau avec une petite table de correspondance qu'on écrit soi-même :

```
noms = pd.DataFrame({
    "denree": ["Millet", "Sorghum", "Maize (local)", "Rice (imported)", "Rice (local)", "Beans
        (niebe)"],
    "nom_fr": ["Mil", "Sorgho", "Maïs local", "Riz importé", "Riz local", "Niébé"],
})
base = pd.merge(recents, noms, on="denree")
print(len(recents), "->", len(base))
base.groupby("nom_fr")["prix"].median().sort_values()
```

```
3686 -> 2235
nom_fr
Maïs local     300.000
Mil            300.000
Sorgho         350.000
Riz importé    370.000
Riz local      425.000
Niébé          879.165
```

3 686 relevés deviennent 2 235 : la jointure interne a retiré toutes les denrées absentes de la table `noms` (arachide, maïs importé…). Ici c'est voulu ; ailleurs, ce serait une perte silencieuse. **Règle** : comparez toujours le nombre de lignes **avant et après** un `merge`.

6.5 Deux pièges, et comment s'en protéger

Le piège de la clé absente. Si les deux tableaux n'ont **aucune** colonne de même nom :

```
try:
    pd.merge(recents, noms.rename(columns={"denree": "produit"}))
except Exception as e:
    print(type(e).__name__, ":", e)
```

```
MergeError : No common columns to perform merge on. Merge options: left_on=None, right_on=None,
left_index=False, right_index=False
```

Si les colonnes ont des noms différents, on les donne séparément : `pd.merge(recents, autre, left_on="denree", right_on="produit")`.

Le piège de la clé en double, plus sournois car il ne donne **aucune erreur**. Si la table `noms` contient deux fois la même denrée, chaque relevé de cette denrée est **dupliqué** :

```
noms_double = pd.concat([noms, noms.head(1)])
try:
    pd.merge(recents, noms_double, on="denree", validate="many_to_one")
except Exception as e:
    print(type(e).__name__, ":", e)
print(len(pd.merge(recents, noms_double, on="denree")), "lignes au lieu de", len(base))
```

```
MergeError : Merge keys are not unique in right dataset; not a many-to-one merge
...
2666 lignes au lieu de 2235
```

Sans protection, 431 lignes de mil en trop (chaque relevé de mil compté deux fois), et toutes les moyennes faussées. `validate="many_to_one"` dit à pandas : « à gauche, la clé peut se répéter (plusieurs relevés de mil) ; à droite, elle doit être **unique** (un seul nom par denrée) ». S'il trouve un doublon, il refuse. Mettez `validate` dès que vous joignez une table de correspondance.

7. Empiler avec `concat`

`merge` colle des tableaux **côte à côte**, en recoupant une clé. `concat` les **empile**, sans rien recouper.

7.1 Des lignes les unes sous les autres

```
r2019 = prix[prix["annee"] == 2019]
r2025 = prix[prix["annee"] == 2025]
deux = pd.concat([r2019, r2025], ignore_index=True)
print(len(r2019), "+", len(r2025), "=", len(deux))
deux["annee"].value_counts()
```

```
2093 + 3686 = 5779
annee
2025    3686
2019    2093
```

Morceau	Ce qu'il veut dire
`pd.concat([r2019, r2025])`	Une **liste** de tableaux, mis bout à bout. Ils doivent avoir les mêmes colonnes ; une colonne absente d'un tableau y est remplie de `NaN`
`ignore_index=True`	Renumérote les lignes de 0 à 5 778. Sans lui, chaque ligne garde son étiquette d'origine

Cas typique : chaque région envoie un fichier par mois ; on les lit dans une liste, puis un seul `pd.concat` les réunit.

7.2 Des colonnes côte à côte

Avec `axis=1`, `concat` place des `Series` **côte à côte**, en les alignant sur leur index :

```
mediane_2019 = r2019.groupby("denree")["prix"].median().rename("2019")
mediane_2025 = r2025.groupby("denree")["prix"].median().rename("2025")
compare = pd.concat([mediane_2019, mediane_2025], axis=1)
compare["hausse_pct"] = ((compare["2025"] / compare["2019"] - 1) * 100).round(1)
compare.dropna().sort_values("hausse_pct", ascending=False)
```

```
                 2019   2025  hausse_pct
denree
Maize (local)     210.0  300.0        42.9
Rice (local)      300.0  425.0        41.7
Sorghum           250.0  350.0        40.0
Maize (imported)  225.0  300.0        33.3
Millet            225.0  300.0        33.3
Rice (imported)   300.0  370.0        23.3
```

Les deux `Series` sont alignées sur la denrée : chaque prix de 2019 se retrouve en face du prix de 2025 de la **même** denrée, même si elles n'étaient pas dans le même ordre. Les denrées présentes une seule année ont un `NaN` dans l'autre colonne : `dropna()` les retire. En six ans, le maïs local a pris 43 %, le riz importé 23 %.

8. Le tableau croisé : `pivot_table`

Vous connaissez peut-être le « tableau croisé dynamique » d'Excel. `pivot_table` fait la même chose : une ligne par valeur d'une colonne, une colonne par valeur d'une autre, et une statistique dans chaque case.

```
cereales = ["Millet", "Sorghum", "Maize (local)", "Rice (imported)"]
selection = recents[recents["denree"].isin(cereales)]
grille = selection.pivot_table(index="region", columns="denree", values="prix", aggfunc="median")
grille.round(0)
```

```
denree       Maize (local)  Millet  Rice (imported)  Sorghum
region
Dakar                300.0   300.0            450.0    350.0
Diourbel             275.0   275.0            375.0    450.0
Fatick               283.0   290.0            375.0    316.0
Kaffrine             200.0   200.0            415.0    250.0
Kaolack              211.0   212.0            360.0    312.0
Kedougou             300.0   425.0            400.0    400.0
Kolda                250.0   300.0            350.0    300.0
Louga                250.0   292.0            370.0    434.0
Matam                350.0   350.0            381.0    400.0
Saint Louis          294.0   300.0            350.0    375.0
Sedhiou              300.0   312.0            350.0    400.0
Tambacounda          293.0   306.0            359.0    300.0
Thies                300.0   300.0            350.0    350.0
Ziguinchor           400.0   400.0            350.0    500.0
```

Morceau	Ce qu'il veut dire
`index="region"`	Une **ligne** par région
`columns="denree"`	Une **colonne** par denrée
`values="prix"`	La colonne dont on calcule la statistique
`aggfunc="median"`	La statistique de chaque case : la médiane des prix de cette région pour cette denrée (par défaut : `"mean"`)

C'est exactement `recents.groupby(["region", "denree"])["prix"].median()`, mais **déplié** : la deuxième clé passe en colonnes. Le résultat se lit d'un coup d'œil : 14 régions, 4 céréales, 56 cases. Une case sans relevé contiendrait `NaN`.

Un tableau croisé s'interroge comme n'importe quel `DataFrame` :

```python
print(grille["Millet"].idxmax(), grille["Millet"].max())
print(grille["Millet"].idxmin(), grille["Millet"].min())
grille.isna().sum()
```

```
Kedougou 425.0
Kaffrine 200.0
denree
Maize (local)      0
Millet             0
Rice (imported)    0
Sorghum            0
```

`idxmax()` donne l'**étiquette** de la plus grande valeur (ici la région), `idxmin()` celle de la plus petite. En 2025, le kilo de mil coûte en médiane 425 F à Kédougou et 200 F à Kaffrine, au cœur du bassin de production : plus du double.

🔵 Avec `margins=True`, `pivot_table` ajoute une ligne et une colonne de **totaux** (ici, la médiane de tous les relevés concernés) :

```
evolution = prix[prix["denree"].isin(cereales) & prix["annee"].between(2019, 2025)].pivot_table(
    index="annee", columns="denree", values="prix", aggfunc="median", margins=True,
         margins_name="Toutes")
evolution.round(0)
```

```
denree  Maize (local)  Millet  Rice (imported)  Sorghum  Toutes
annee
2019            210.0   225.0            300.0    250.0   250.0
2020            220.0   264.0            325.0    250.0   275.0
2021            248.0   250.0            325.0    250.0   275.0
2022            350.0   400.0            350.0    400.0   363.0
2023            350.0   425.0            350.0    400.0   363.0
2024            300.0   325.0            425.0    360.0   350.0
2025            300.0   300.0            370.0    350.0   328.0
Toutes          269.0   300.0            350.0    312.0   310.0
```

`between(2019, 2025)` garde les années de 2019 à 2025, bornes comprises. Le saut de 2022 est net : le mil passe de 250 à 400 F (+60 %), l'année où les prix mondiaux des céréales et des engrais ont flambé. Le riz importé, lui, monte surtout en 2024.

> 🧨 **Essayez** : `recents.pivot(index="region", columns="denree", values="prix")` (`pivot` sans `_table`). Message : `ValueError: Index contains duplicate entries, cannot reshape`. `pivot` ne fait que **déplacer** les valeurs : il exige **une seule** valeur par case. Ici, chaque région a plusieurs relevés de mil (plusieurs marchés, douze mois) : il faut `pivot_table`, qui les **résume** avec `aggfunc`. Annulez.

9. Format large et format long : `melt`

La grille de la section 8 est au **format large** : une colonne par denrée. C'est idéal pour lire. Mais beaucoup d'outils (les graphiques de seaborn au chapitre 6, les bases de données, `groupby`) préfèrent le **format long** : une ligne par observation, avec une colonne qui dit **de quelle denrée** il s'agit.

Format large (14 lignes, 5 colonnes)
`region | Mil | Sorgho | ...`
`Dakar | 300 | 350 | ...`

⇄

Format long (56 lignes, 3 colonnes)
`region | denree | prix_median`
`Dakar | Mil | 300`
`Dakar | Sorgho | 350`

`melt` (« faire fondre ») passe du large au long :

```
large = grille.round(0).reset_index()
print(large.shape)
long = large.melt(id_vars="region", var_name="denree", value_name="prix_median")
print(long.shape)
long.head(4)
```

```
(14, 5)
(56, 3)
     region         denree  prix_median
0     Dakar  Maize (local)        300.0
1  Diourbel  Maize (local)        275.0
2    Fatick  Maize (local)        283.0
3  Kaffrine  Maize (local)        200.0
```

Morceau	Ce qu'il veut dire
`grille.round(0).reset_index()`	La région, qui était l'index, redevient une colonne ordinaire
`id_vars="region"`	La colonne qui **reste** telle quelle et se répète
`var_name="denree"`	Le nom de la nouvelle colonne qui reçoit les **anciens noms de colonnes** (Mil, Sorgho...)
`value_name="prix_median"`	Le nom de la colonne qui reçoit les **valeurs** des cases
14 x 4 = 56 lignes	Une ligne par case de la grille

Et `pivot` fait le chemin inverse, du long au large. Ici, chaque couple (région, denrée) n'a qu'une valeur : `pivot` suffit.

```
retour = long.pivot(index="region", columns="denree", values="prix_median")
retour.equals(grille.round(0))
```

```
True
```

`equals` compare deux tableaux case par case : l'aller-retour redonne exactement la grille.

🟢 **Quel format choisir ?** Large pour **lire** et présenter (un rapport, un tableau à imprimer) ; long pour **calculer**, filtrer et tracer. Les fichiers qu'on reçoit sont souvent au format large (une colonne par mois, par année) : `melt` est alors la première étape.

Exercices

Ouvrez un notebook `exercices-ch5`.

- ★ **Exercice 1 : la couverture du PAM, année par année.** Sur les relevés au kilo depuis 2020, construisez avec une agrégation nommée un tableau d'une ligne par année : nombre de relevés, nombre de marchés différents, nombre de denrées différentes. Quelle année a le plus de marchés ? le

plus de relevés ? Attendu : 53 marchés en 2020 ; 4 502 relevés en 2024 ; 25 denrées en 2026. (Sections 3 et 4.)

- ★ **Exercice 2 : les notes.** Créez `notes = pd.DataFrame({"etudiant": ["Awa", "Awa", "Awa", "Moussa", "Moussa", "Moussa", "Khady", "Khady", "Khady"], "matiere": ["Maths", "Info", "Anglais"] * 3, "note": [12, 16, 14, 9, 13, 11, 15, 18, 10]})`. Calculez la moyenne de chaque étudiant (classement), puis le minimum, le maximum et la moyenne de chaque matière en une seule instruction. Ajoutez une colonne : l'écart de chaque note à la moyenne de **sa** matière. Qui a le plus grand écart positif, et dans quelle matière ? (Sections 3 à 5.)
- ★★ **Exercice 3 : la bibliothèque.** Créez deux tableaux. `livres` : `id_livre` 1 à 5 ; titres *Une si longue lettre*, *L'Aventure ambiguë*, *Les Bouts de bois de Dieu*, *Le Ventre de l'Atlantique*, *Xala* ; auteurs Mariama Bâ, Cheikh Hamidou Kane, Ousmane Sembène, Fatou Diome, Ousmane Sembène. `emprunts` : `id_livre` 1, 1, 3, 5, 1, 3, 6 ; lecteurs Binta, Ousseynou, Binta, Mame, Aliou, Mame, Aliou. Combien d'emprunts concernent un livre connu ? Quel auteur est le plus emprunté ? Quels livres n'ont jamais été empruntés (jointure `left`) ? Avec `how="outer"` et `indicator=True`, que révèle la ligne `right_only` ? (Section 6.)
- ★★ **Exercice 4 : la pluie.** Voici des cumuls de pluie mensuels **fictifs** (en mm), au format large : `pluie = pd.DataFrame({"ville": ["Ziguinchor", "Kaolack", "Saint-Louis"], "juin": [120, 40, 5], "juillet": [330, 130, 30], "aout": [480, 210, 90], "septembre": [380, 160, 100], "octobre": [150, 40, 10]})`. Passez-le au format long avec `melt` (combien de lignes ?), calculez le cumul de la saison par ville, puis trouvez le mois le plus pluvieux de chaque ville (indice : `idxmax` sur un `groupby` donne les étiquettes des lignes, à passer à `loc`). Revenez au format large avec `pivot` : dans quel ordre sont les mois, et comment le corriger ? (Sections 3 et 9.)
- ★★★ **Exercice 5 : le marché le plus régulier.** Pour le riz importé en 2025, calculez par marché le nombre de relevés, la moyenne et l'écart-type des prix (`"std"`). Ne gardez que les marchés qui ont au moins 10 relevés, puis calculez le **coefficient de variation** (écart-type divisé par la moyenne, en %). Quel marché a les prix les plus stables ? les plus variables ? Attendu : 27 marchés ; le plus stable, Porokhane (4,4 %). 🟣 Retrouvez le même filtrage avec `groupby("market").filter(lambda g: len(g) >= 10)`, qui garde les **lignes** des groupes qui remplissent la condition : combien de relevés restent ? (Sections 4 et 5.)

Atelier 5 : les indicateurs par région et par denrée

Le travail principal du chapitre : le Commissariat à la sécurité alimentaire vous demande une note sur les prix de 2025 : quelles régions sont les plus chères, pour quelles denrées, comment les prix ont évolué depuis 2019, où la soudure pèse le plus, et quels marchés ne sont plus suivis. Vous construisez ces indicateurs avec `groupby`, `agg`, `transform`, `merge`, `pivot_table` et `melt`, puis vous les enregistrez dans un classeur Excel de plusieurs feuilles. L'énoncé et le notebook de départ vous sont fournis séparément.

A retenir

- `df.groupby("cle")["colonne"].statistique()` : diviser, appliquer, combiner. Plusieurs clés : une liste ; `as_index=False` ou `reset_index()` pour un tableau ordinaire.

- Toujours regarder **combien de lignes** il y a dans chaque groupe (`size`) : une moyenne sur deux relevés ne vaut rien.
- `agg(["mean", "median"])` pour plusieurs statistiques ; `agg(nom=("colonne", "statistique"))` pour des noms clairs ; une fonction à soi passée sans parenthèses.
- `transform` recopie la statistique du groupe sur chaque ligne : pour **comparer** une ligne à son groupe.
- `pd.merge(a, b, on="cle", how=...)` : `inner` (par défaut) perd en silence ce qui ne correspond pas ; `indicator=True` montre ce qui s'est passé ; `validate="many_to_one"` protège des doublons ; comparer le nombre de lignes avant et après.
- `pd.concat([a, b])` empile des lignes (`ignore_index=True` pour renuméroter) ; `axis=1` aligne des `Series` côte à côte sur leur index.
- `pivot_table(index=, columns=, values=, aggfunc=)` : le tableau croisé ; `pivot` seulement s'il y a une seule valeur par case.
- `melt` : du large au long (pour calculer et tracer) ; `pivot` : du long au large (pour lire).

Erreurs fréquentes

Message ou symptôme	Ce que ça veut dire	Solution
`KeyError: 'regions'` dans un `groupby`	Colonne inexistante	Vérifier `df.columns.tolist()`
`TypeError: dtype 'str' does not support operation 'mean'`	Statistique numérique sur une colonne de texte	Choisir la bonne colonne après le `groupby` , ou convertir (chapitre 4)
`AttributeError: 'SeriesGroupBy' object has no attribute 'moyenne'`	Nom de statistique inconnu dans `agg`	Les noms anglais : `"mean"` , `"median"` , `"sum"` , `"size"` , `"nunique"` …
Une moyenne par groupe surprenante (le fonio en tête)	Groupe minuscule	Afficher `size()` à côté
Des lignes disparaissent après un `merge` , sans erreur	Jointure interne : les clés sans correspondance sont retirées	`how="left"` et `indicator=True` ; compter avant et après
Plus de lignes après un `merge` qu'avant	Clé en double dans la table de droite : des lignes dupliquées	`validate="many_to_one"` ; `drop_duplicates` sur la table de correspondance
`MergeError: No common columns to perform merge on`	Pas de colonne de même nom	`on=` avec le bon nom, ou `left_on=` et `right_on=`
`MergeError: Merge keys are not unique in right dataset`	`validate` a trouvé un doublon : c'est une **protection**	Corriger la table de droite
`ValueError: Index contains duplicate entries, cannot reshape`	`pivot` avec plusieurs valeurs par case	`pivot_table` avec `aggfunc`

Message ou symptôme	Ce que ça veut dire	Solution
`NaN` dans un tableau croisé	Aucun relevé pour cette combinaison	Normal ; le signaler, ou `fill_value=` si un 0 a un sens (un comptage)
Les colonnes de `pivot` sont dans l'ordre alphabétique	`pivot` trie les étiquettes	Réordonner : `df[["juin", "juillet", ...]]`

Pour aller plus loin

- Le guide officiel de `groupby` (en anglais) : https://pandas.pydata.org/docs/user_guide/groupby.html
- Le guide officiel de `merge`, `concat` et des jointures (en anglais) : https://pandas.pydata.org/docs/user_guide/merging.html
- Le guide officiel de `pivot_table`, `melt` et des restructurations (en anglais) : https://pandas.pydata.org/docs/user_guide/reshaping.html
- La comparaison avec SQL, utile si vous connaissez les bases de données : https://pandas.pydata.org/docs/getting_started/comparison/comparison_with_sql.html